Trabajo Práctico - Programa dron bombardero
Algoritmos y Estructura de Datos K1153

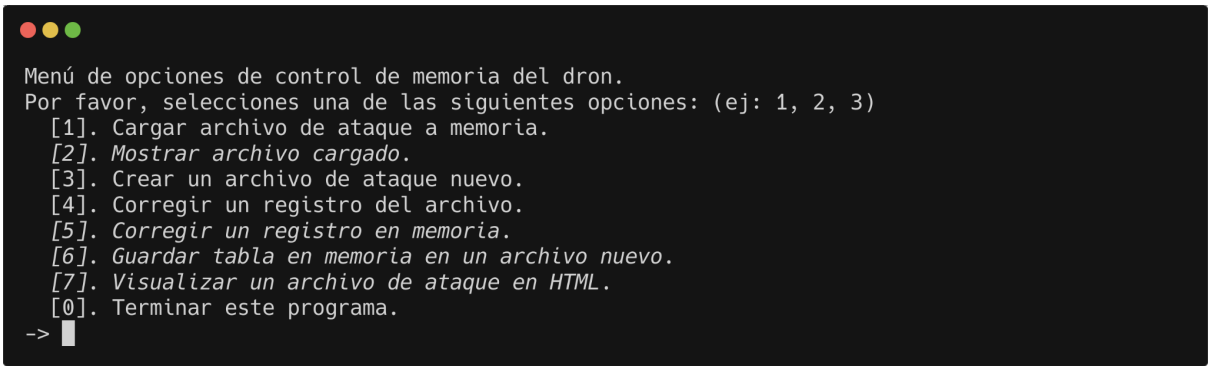
Miembros del equipo:
Desarrollador principal: Joaquín Kainer - 233.968-7 - github.com/jkainer - jkainer@frba.utn.edu.ar

Índice

- Guía de uso
  - Crear de una tabla de órdenes
  - Cargar una tabla de órdenes
  - Mostrar una tablas de órdenes
  - Modificar un registro de una tabla en memoria
  - Guardar tabla en memoria en un archivo
  - Modificar un registro de una tabla en archivo
  - Visualizar una tablas de órdenes mediante HTML
- Condiciones de validación de una tabla de órdenes
- Funcionamiento y desarrollo del programa
  - *Lista a tabla* y *Tabla a lista*
  - *Clonar tabla*
  - *Bucle principal*
  - *Cargar archivo*
  - *Mostrar tabla*
  - *Crear ataque*
  - *Modificar registro*
  - *Modificar registro de archivo*
  - *Guardar ataque*
  - *Crear HTML*
- Uso de Inteligencia Artificial
- Agradecimientos
- Referencias

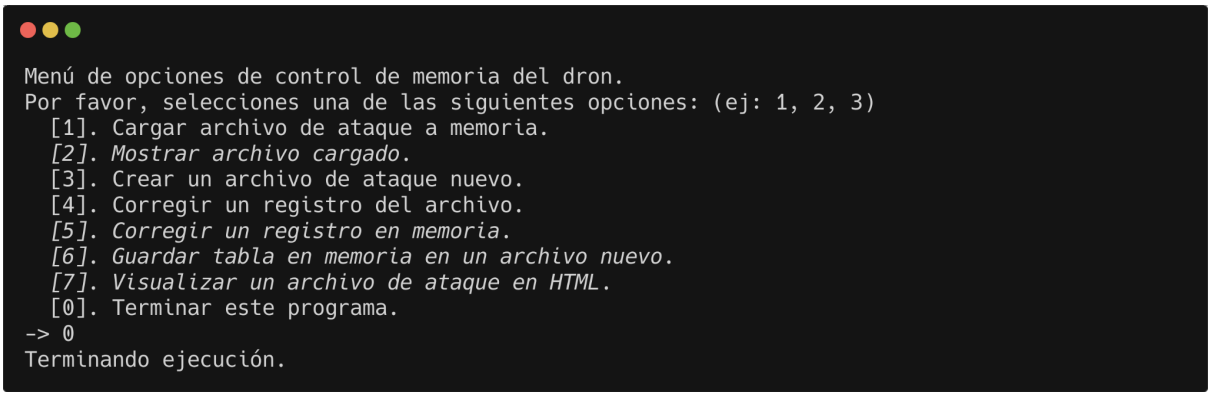
Guía de uso

> Durante las pruebas y desarrollo del programa junto a la realización de las capturas de ejemplo de la siguiente guía, se utilizó una versión modificada del programa para trabajar con tablas de resolución 5×5.

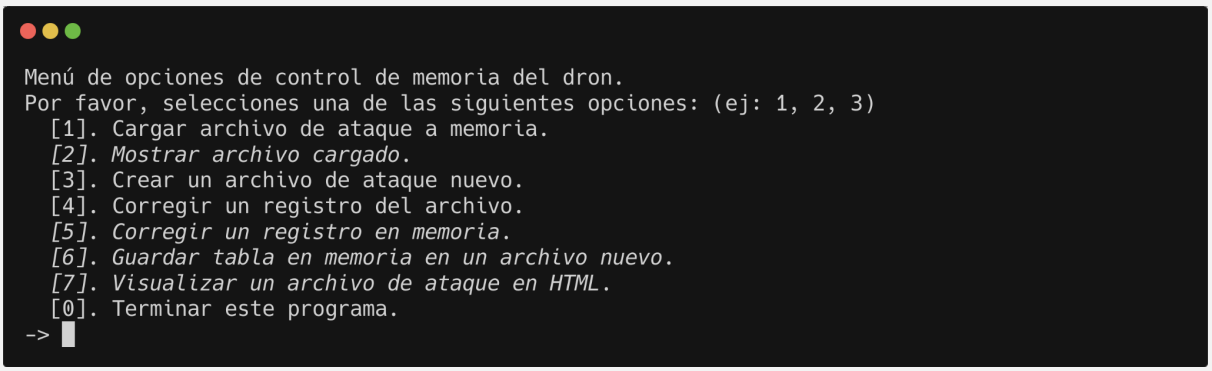


Al iniciar, el programa presenta un menú principal con 7 funciones distintas, identificadas por un número de acceso.

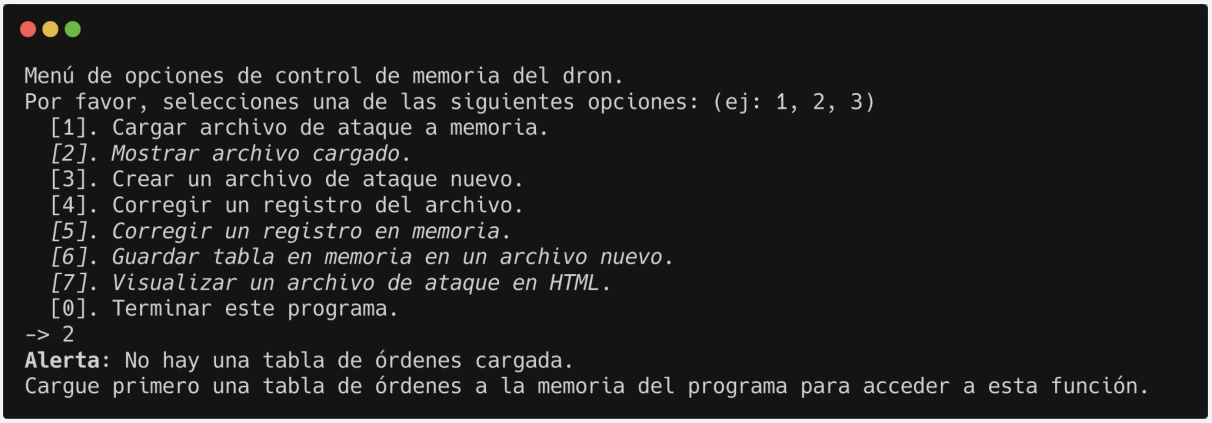
Para acceder a cualquiera de las opciones del menú se debe introducir el número correspondiente, o `[0]` para terminar la ejecución.



Apenas iniciado el programa, ciertas opciones del menú están resaltadas en itálica, esto porque se encuentran deshabilitadas ya que son funciones que requieren de una tabla de órdenes cargada en memoria para su ejecución.



Si se intenta acceder a una de estas opciones sin una tabla cargada, ocurre la siguiente alerta, devolviendo al menú principal de regreso.



Crear de una tabla de órdenes

Para crear una tabla, se accede a la opción `[3]` desde el menú principal.

Uno por uno, el programa solicitará los registros necesarios para poblar las celdas de la tabla.

```
Posición (2;5):
  ¿Aterrizaje? [s/N]
  ->
  ¿Ataque kamikaze? [s/N]
  ->
  Tiempo de espera (Segundos).
  -> 2
  Posición siguiente (X;Y).
  -> X: 2
  -> Y: 4
Posición (2;4):
  ¿Aterrizaje? [s/N]
  ->
  ¿Ataque kamikaze? [s/N]
  ->
  Tiempo de espera (Segundos).
  -> 555
  Posición siguiente (X;Y).
  -> X: 3
  -> Y: 4
Posición (3;4):
  ¿Aterrizaje? [s/N]
  ->
  ¿Ataque kamikaze? [s/N]
  ->
  Tiempo de espera (Segundos).
  -> 2
  Posición siguiente (X;Y).
  -> X: 3
  -> Y: 4
Error: Seleccione una celda adyasente.
  Posición siguiente (X;Y).
  -> X: 3
  -> Y: 5
```

> Cuando el programa solicite un valor de "Sí" o "No", `[s/n]`, al responder en blanco presionando la tecla Enter el programa asumirá el valor predeterminado, determinado por la mayúscula en la solicitud de respuesta.

	Responde `s`	Responde `n`	Responde `‘ ‘`
`[s/N]`	Respuesta = `s`	Respuesta = `n`	Respuesta = `n`
`[S/n]`	Respuesta = `s`	Respuesta = `n`	Respuesta = `s`

Una vez asignados los valores correspondientes de cada celda de la tabla, el programa procederá a realizar una validación de los datos ingresados.

El programa solicitará la dirección donde guardar el archivo a crear. Si el archivo ya existe, el programa indicará esto y solicitará como continuar. El usuario podrá elegir si sobreescribir el archivo o elegir otra dirección.

```
Por favor, ingrese la ruta donde guardar el archivo.
-> demo/archivo.tabla
Alerta: El archivo ya existe. ¿Desea sobreescribirlo? [s/N]
-> s
Guardando tabla de órdenes en "demo/archivo.tabla"…
Archivo guardado correctamente.
```

> Se recomienda diseñar con antelación la tabla a ingresar en un medio gráfico como papel, ya que el programa presenta una interfaz de texto, y puede resultar difícil de asignar una ruta sin una guía visual de auxilio.

Cargar una tabla de órdenes

Para cargar una tabla de órdenes a la memoria, se accede a la opción `[1]` desde el menú principal.

El programa solicitará la dirección del archivo donde se almacena la tabla de órdenes.

```
Menú de opciones de control de memoria del dron.
Por favor, selecciones una de las siguientes opciones: (ej: 1, 2, 3)
  [1]. Cargar archivo de ataque a memoria.
  [2]. Mostrar archivo cargado.
  [3]. Crear un archivo de ataque nuevo.
  [4]. Corregir un registro del archivo.
  [5]. Corregir un registro en memoria.
  [6]. Guardar tabla en memoria en un archivo nuevo.
  [7]. Visualizar un archivo de ataque en HTML.
  [0]. Terminar este programa.
-> 1
Por favor, ingrese una ruta de un archivo a cargar.
-> demo/archivo.tabla
Cargando archivo de ataque "demo/archivo.tabla"…
Archivo cargado correctamente.
Presione Enter para continuar…
```

El programa entonces procederá a realizar una validación de los datos de la tabla. En caso de que la tabla sea inválida, el programa volverá a solicitar la dirección de un archivo que contenga una tabla válida.

```
Resultado:
┌──────────────────────────────────┐
│ Posición actual (X;Y):      (1;2)│
│ Despegue:                Veradero│
│ Aterrizaje:                Falso │
│ Soltar granada nº1:        Falso │
│ Soltar granada nº2:        Falso │
│ Ataque kamikaze:           Falso │
│ Espera [Segundos]:          80 s │
│ Posición siguiente (X;Y):   (1;3)│
└──────────────────────────────────┘
¿Cambiar otro registro? [s/N]
->
¿Guardar cambios? [S/n]
->
Validando cambios…
Error: La tabla contiene más de 1 celda de despegue.
Error: Las modificaciones hechas a la tabla son inválidas. Deshaciendo los cambios.
```

Una vez cargado correctamente el archivo, la tabla que contenía ahora se encuentra almacenada en la memoria del programa. Esto se puede comprobar en el menú principal, que ahora indica el nombre del archivo cargado.

```
Menú de opciones de control de memoria del dron.
Archivo cargado: "demo/archivo.tabla"
Por favor, selecciones una de las siguientes opciones: (ej: 1, 2, 3)
  [1]. Cargar archivo de ataque a memoria.
  [2]. Mostrar archivo cargado.
  [3]. Crear un archivo de ataque nuevo.
  [4]. Corregir un registro del archivo.
  [5]. Corregir un registro en memoria.
  [6]. Guardar tabla en memoria en un archivo nuevo.
  [7]. Visualizar un archivo de ataque en HTML.
  [0]. Terminar este programa.
-> 
```

Mostrar una tablas de órdenes

> Esta función solo está habilitada con una tabla de órdenes ya cargada en memoria. De lo contrario, acceder a ella devolverá una alerta.

```
Menú de opciones de control de memoria del dron.
Por favor, selecciones una de las siguientes opciones: (ej: 1, 2, 3)
  [1]. Cargar archivo de ataque a memoria.
  [2]. Mostrar archivo cargado.
  [3]. Crear un archivo de ataque nuevo.
  [4]. Corregir un registro del archivo.
  [5]. Corregir un registro en memoria.
  [6]. Guardar tabla en memoria en un archivo nuevo.
  [7]. Visualizar un archivo de ataque en HTML.
  [0]. Terminar este programa.
-> 2
Alerta: No hay una tabla de órdenes cargada.
Cargue primero una tabla de órdenes a la memoria del programa para acceder a esta función.
```

Para visualizar los registros que componen a una tabla de órdenes cargada previamente en memoria, se accede a la opción `[2]` desde el menú principal.

El programa mostrará los contenidos de cada registro de la tabla cargada de izquierda a derecha, de arriba a abajo, desde la celda 1×1 hasta la 200×200.

```
┌──────────────────────────────────┐
│ Posición actual (X;Y):     (3;2) │
│ Despegue:                 Falso  │
│ Aterrizaje:               Falso  │
│ Soltar granada nº1:       Falso  │
│ Soltar granada nº2:       Falso  │
│ Ataque kamikaze:          Falso  │
│ Espera [Segundos]:         10 s  │
│ Posición siguiente (X;Y):  (3;1) │
└──────────────────────────────────┘
┌──────────────────────────────────┐
│ Posición actual (X;Y):     (4;2) │
│ Despegue:                Veradero│
│ Aterrizaje:               Falso  │
│ Soltar granada nº1:       Falso  │
│ Soltar granada nº2:       Falso  │
│ Ataque kamikaze:          Falso  │
│ Espera [Segundos]:         25 s  │
│ Posición siguiente (X;Y):  (4;1) │
└──────────────────────────────────┘
┌──────────────────────────────────┐
│ Posición actual (X;Y):     (5;2) │
│ Despegue:                 Falso  │
│ Aterrizaje:               Falso  │
│ Soltar granada nº1:       Falso  │
│ Soltar granada nº2:       Falso  │
│ Ataque kamikaze:          Falso  │
│ Espera [Segundos]:          8 s  │
│ Posición siguiente (X;Y):  (5;3) │
└─ Presione Enter para continuar… ─┘
```

Una vez finalizado el recorrido, el programa volverá al menú principal.

Modificar un registro de una tabla en memoria

> Esta función solo está habilitada con una tabla de órdenes ya cargada en memoria. De lo contrario, acceder a ella devolverá una alerta.

```
Menú de opciones de control de memoria del dron.
Por favor, selecciones una de las siguientes opciones: (ej: 1, 2, 3)
  [1]. Cargar archivo de ataque a memoria.
  [2]. Mostrar archivo cargado.
  [3]. Crear un archivo de ataque nuevo.
  [4]. Corregir un registro del archivo.
  [5]. Corregir un registro en memoria.
  [6]. Guardar tabla en memoria en un archivo nuevo.
  [7]. Visualizar un archivo de ataque en HTML.
  [0]. Terminar este programa.
-> 2
Alerta: No hay una tabla de órdenes cargada.
Cargue primero una tabla de órdenes a la memoria del programa para acceder a esta función.
```

Para modificar un registro de una tabla de órdenes cargada previamente en memoria, se accede a la opción `[5]` desde el menú principal.

```
Menú de opciones de control de memoria del dron.
Archivo cargado: "demo/archivo.tabla"
Por favor, selecciones una de las siguientes opciones: (ej: 1, 2, 3)
  [1]. Cargar archivo de ataque a memoria.
  [2]. Mostrar archivo cargado.
  [3]. Crear un archivo de ataque nuevo.
  [4]. Corregir un registro del archivo.
  [5]. Corregir un registro en memoria.
  [6]. Guardar tabla en memoria en un archivo nuevo.
  [7]. Visualizar un archivo de ataque en HTML.
  [0]. Terminar este programa.
-> 5
¿Qué registro desea modificar? (X;Y)
-> X: 5
-> Y: 5
Registro original:
┌──────────────────────────────────┐
│ Posición actual (X;Y):      (5;5)│
│ Despegue:                  Falso │
│ Aterrizaje:                Falso │
│ Soltar granada nº1:        Falso │
│ Soltar granada nº2:        Falso │
│ Ataque kamikaze:         Veradero│
│ Espera [Segundos]:           3 s │
│ Posición siguiente (X;Y):  Final │
└──────────────────────────────────┘
Posición (5;5):
  ¿Despegue? [s/N]
  ->
  ¿Aterrizaje? [s/N]
  -> s
  ¿Soltar granada nº1? [s/N]
  ->
  ¿Soltar granada nº2? [s/N]
  ->
  Tiempo de espera (Segundos).
  -> 15

Resultado:
┌──────────────────────────────────┐
│ Posición actual (X;Y):      (5;5)│
│ Despegue:                  Falso │
│ Aterrizaje:              Veradero│
│ Soltar granada nº1:        Falso │
│ Soltar granada nº2:        Falso │
│ Ataque kamikaze:           Falso │
│ Espera [Segundos]:          15 s │
│ Posición siguiente (X;Y):  Final │
└──────────────────────────────────┘
¿Cambiar otro registro? [s/N]
->
¿Guardar cambios? [S/n]
->
Validando cambios…
Cambios validados. Todo en orden.
```

El programa solicitará las coordenadas de la celda a modificar. Una vez elegida la celda, se mostrará por pantalla el registro original.

Aplicados los cambios respectivos, el programa mostrará el registro modificado con los cambios hechos, y solicitará como continuar.

El usuario puede elegir si modificar otro registro. Entonces, el programa solicitará las coordenadas de la celda nueva, y se repetirá el proceso.

Una vez finalizados los cambios, el programa solicitará si guardar los cambios ejercidos en el archivo. Entonces, el programa procederá a realizar una validación de los datos de la tabla

con los nuevos cambios aplicados. En caso de que los cambios sean inválidos, el programa desechará los cambios hechos, volviendo al menú principal.

Con los cambios validados, la tabla modificada se guardará en la memoria del programa.

> El programa indicará que se han realizado cambios a la tabla de órdenes sin guardar en el menú principal con el nombre del archivo cargado en itálica y con un asterisco hasta que se guarden los datos.

```
Menú de opciones de control de memoria del dron.
Archivo cargado: "demo/archivo.tabla"*
Por favor, selecciones una de las siguientes opciones: (ej: 1, 2, 3)
  [1]. Cargar archivo de ataque a memoria.
  [2]. Mostrar archivo cargado.
  [3]. Crear un archivo de ataque nuevo.
  [4]. Corregir un registro del archivo.
  [5]. Corregir un registro en memoria.
  [6]. Guardar tabla en memoria en un archivo nuevo.
  [7]. Visualizar un archivo de ataque en HTML.
  [0]. Terminar este programa.
-> 
```

Guardar tabla en memoria en un archivo

> Esta función solo está habilitada con una tabla de órdenes ya cargada en memoria. De lo contrario, acceder a ella devolverá una alerta.

```
Menú de opciones de control de memoria del dron.
Por favor, selecciones una de las siguientes opciones: (ej: 1, 2, 3)
  [1]. Cargar archivo de ataque a memoria.
  [2]. Mostrar archivo cargado.
  [3]. Crear un archivo de ataque nuevo.
  [4]. Corregir un registro del archivo.
  [5]. Corregir un registro en memoria.
  [6]. Guardar tabla en memoria en un archivo nuevo.
  [7]. Visualizar un archivo de ataque en HTML.
  [0]. Terminar este programa.
-> 2
Alerta: No hay una tabla de órdenes cargada.
Cargue primero una tabla de órdenes a la memoria del programa para acceder a esta función.
```

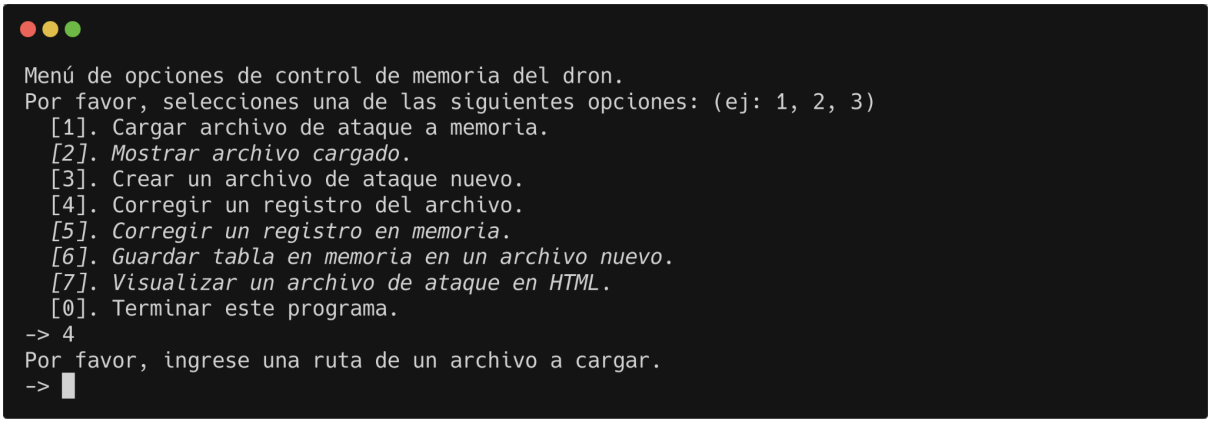
Para guardar una tabla de órdenes en memoria en un archivo, se accede a la opción `[6]` desde el menú principal.

El programa solicitará la dirección donde guardar el archivo. Si el archivo ya existe, el programa indicará esto y solicitará como continuar. El usuario podrá elegir si sobreescribir el archivo o elegir otra dirección.

```
Por favor, ingrese la ruta donde guardar el archivo.
-> demo/archivo.tabla
Alerta: El archivo ya existe. ¿Desea sobreescribirlo? [s/N]
-> s
Guardando tabla de órdenes en "demo/archivo.tabla"…
Archivo guardado correctamente.
```

Modificar un registro de una tabla en archivo

Para modificar un registro de una tabla de órdenes de un archivo, se accede a la opción `[4]` desde el menú principal.



El programa solicitará las coordenadas de la celda a modificar. Una vez elegida la celda, se mostrará por pantalla el registro original.

Aplicados los cambios respectivos, el programa mostrará el registro con los cambios hechos, y solicitará como continuar.

El usuario puede elegir si modificar otro registro. Entonces, el programa solicitará las coordenadas de la celda nueva, y se repetirá el proceso.

Una vez finalizados los cambios, el programa solicitará si guardar los cambios ejercidos en el archivo. Entonces, el programa procederá a realizar una validación de los datos de la tabla con los nuevos cambios aplicados. En caso de que los cambios sean inválidos, el programa desechará los cambios hechos, volviendo al menú principal.

Con los cambios validados, la tabla modificada se guardará en el archivo original, sobreescribiendo los registros originales.

Visualizar una tablas de órdenes mediante HTML

Esta función solo está habilitada con una tabla de órdenes ya cargada en memoria. De lo contrario, acceder a ella devolverá una alerta.

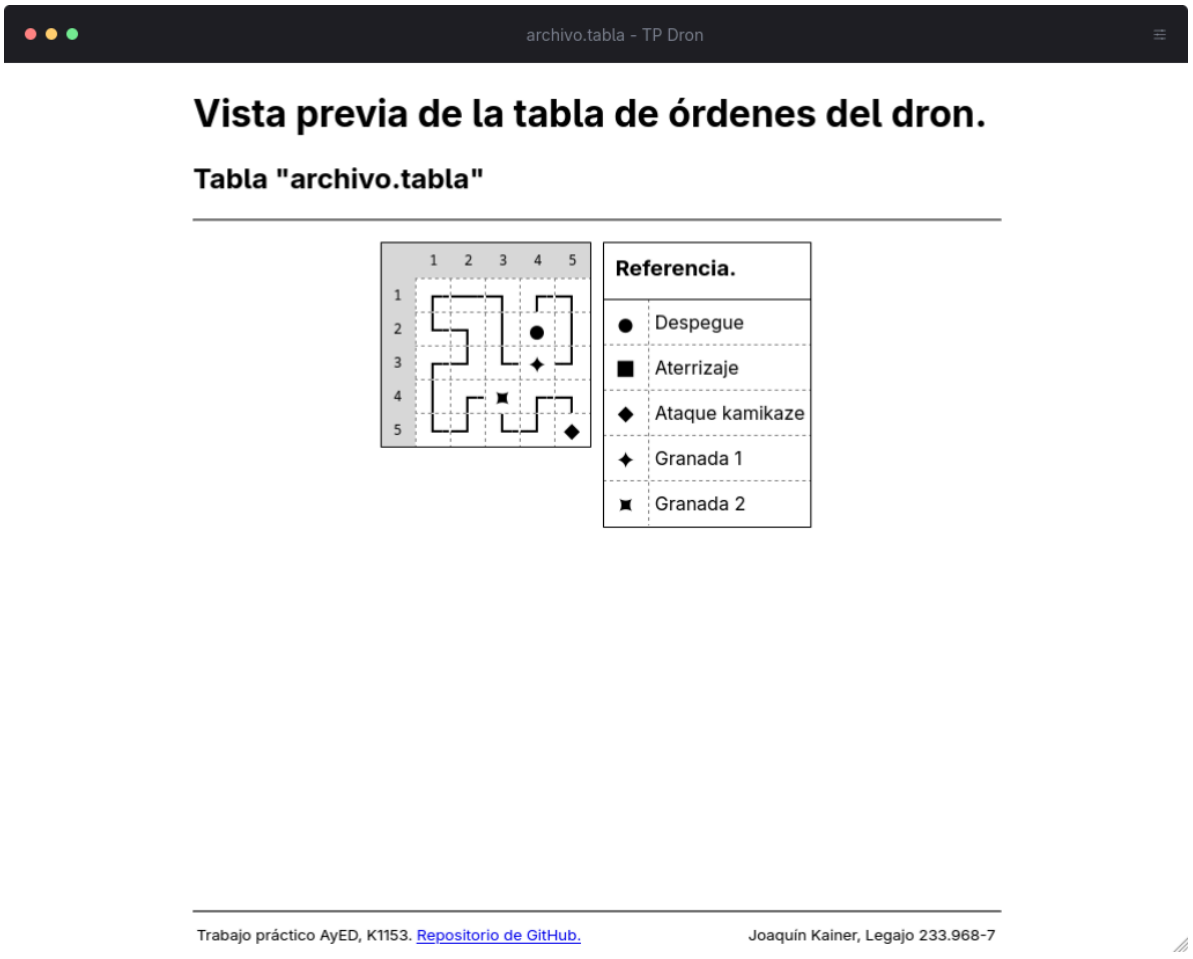
```
Menú de opciones de control de memoria del dron.
Por favor, selecciones una de las siguientes opciones: (ej: 1, 2, 3)
  [1]. Cargar archivo de ataque a memoria.
  [2]. Mostrar archivo cargado.
  [3]. Crear un archivo de ataque nuevo.
  [4]. Corregir un registro del archivo.
  [5]. Corregir un registro en memoria.
  [6]. Guardar tabla en memoria en un archivo nuevo.
  [7]. Visualizar un archivo de ataque en HTML.
  [0]. Terminar este programa.
-> 2
Alerta: No hay una tabla de órdenes cargada.
Cargue primero una tabla de órdenes a la memoria del programa para acceder a esta función.
```

Para visualizar una tabla de órdenes en memoria mediante una página HTML, se accede a la opción `[7]` desde el menú principal.

El programa generará entonces una página web HTML en el directorio del archivo cargado.

```
Menú de opciones de control de memoria del dron.
Archivo cargado: "demo/archivo.tabla"
Por favor, selecciones una de las siguientes opciones: (ej: 1, 2, 3)
  [1]. Cargar archivo de ataque a memoria.
  [2]. Mostrar archivo cargado.
  [3]. Crear un archivo de ataque nuevo.
  [4]. Corregir un registro del archivo.
  [5]. Corregir un registro en memoria.
  [6]. Guardar tabla en memoria en un archivo nuevo.
  [7]. Visualizar un archivo de ataque en HTML.
  [0]. Terminar este programa.
-> 7
El archivo "demo/archivo.tabla.html" se guardó correctamente.
Presione Enter para continuar…
```

Se puede acceder a este archivo HTML con cualquier navegador moderno.



Condiciones de validación de una tabla de órdenes

La lista de condiciones que una tabla debe cumplir para ser reconocida como válida son las siguientes:

- Tener exactamente 1 celda de despegue.
  - Una celda de despegue no puede contener ninguna otra instrucción.
- Tener exactamente 1 celda de finalización (Aterrizaje o kamikaze).
  - No tener simultáneamente una celda de aterrizaje y una de ataque kamikaze.
- Ser recorrida en su totalidad desde la celda de despegue hasta la de finalización.
  - Recorrer cada celda de la tabla exactamente 1 vez.
  - No recorrer celdas fuera de los límites (≤0 o >200).
  - Solo recorrer celdas adyacentes unas de otras.
- Tener no más de 1 celda de granada nº1 y 1 de granada nº2.

En caso de no cumplir tan solo una de estas condiciones, el programa marcará como inválida la tabla, resultando en un error.

Lista a tabla y *Tabla a lista*

C/C++

```
// Copia el contenido de una lista de prototipo "OrdenArchivo"
// a una tabla en memoria de prototipo "Orden".
void listaATabla(
  OrdenArchivo lista[TABLA_TAMANO_X * TABLA_TAMANO_Y],
  Orden tabla[TABLA_TAMANO_X][TABLA_TAMANO_Y]
) {
  int x, y;
  for (int i = 0; i < TABLA_TAMANO_X * TABLA_TAMANO_Y; i++) {
    x = lista[i].x;
    y = lista[i].y;

    tabla[x][y].despegue        = lista[i].despegue;
    tabla[x][y].aterrizaje      = lista[i].aterrizaje;
    tabla[x][y].soltar_granada1 = lista[i].soltar_granada1;
    tabla[x][y].soltar_granada2 = lista[i].soltar_granada2;
    tabla[x][y].ataque_kamikaze = lista[i].ataque_kamikaze;
    tabla[x][y].espera          = lista[i].espera;
    tabla[x][y].siguiente_x     = lista[i].siguiente_x;
    tabla[x][y].siguiente_y     = lista[i].siguiente_y;
  }

  return;
}
```

C/C++

```
// Copia el contenido de una tabla en memoria de prototipo "Orden"
// a una lista de prototipo "OrdenArchivo".
void tablaALista(
  OrdenArchivo lista[TABLA_TAMANO_X * TABLA_TAMANO_Y],
  Orden tabla[TABLA_TAMANO_X][TABLA_TAMANO_Y]
) {
  int i = 0;
  for (int y = 0; y < TABLA_TAMANO_Y; y++) {
    for (int x = 0; x < TABLA_TAMANO_X; x++) {
      lista[i].x = x;
      lista[i].y = y;

      lista[i].despegue        = tabla[x][y].despegue;
      lista[i].aterrizaje      = tabla[x][y].aterrizaje;
      lista[i].soltar_granada1 = tabla[x][y].soltar_granada1;
      lista[i].soltar_granada2 = tabla[x][y].soltar_granada2;
      lista[i].ataque_kamikaze = tabla[x][y].ataque_kamikaze;
      lista[i].espera          = tabla[x][y].espera;
      lista[i].siguiente_x     = tabla[x][y].siguiente_x;
      lista[i].siguiente_y     = tabla[x][y].siguiente_y;
      i++;
    }
  }
```

```
  return;
}
```

El par de procedimientos `listaATabla()` y `tablaALista()` tienen por función el de poblar los registros de una lista dada por parámetro a una tabla también dada por parámetro y viceversa.

Clonar tabla

```C/C++
// Copia el contenido de una tabla de órdenes "Original" a otra "Copia".
void clonarTabla(
  Orden original[TABLA_TAMANO_X][TABLA_TAMANO_Y],
  Orden copia[TABLA_TAMANO_X][TABLA_TAMANO_Y]
) {
  // Podría usar un solo bucle For y aritmética de punteros para transitar
  // las tablas, pero en pro de la legibilidad, prefiero recorrerlo en
  // ambos ejes por separado con dos bucles.
  for (int y = 0; y < TABLA_TAMANO_Y; y++) {
    for (int x = 0; x < TABLA_TAMANO_X; x++) {
      copia[x][y] = original[x][y];
    }
  }

  return;
}
```

Esta función copia el contenido de una tabla dada por parámetro a otra también dada por parámetro registro a registro.

Bucle principal

El programa se ejecuta bajo un bucle principal, que comienza mostrando en pantalla el "menú principal", y solicitando al usuario un valor numérico.

```C/C++
// Menú principal.
void bucleMenu(
  Orden tabla[TABLA_TAMANO_X][TABLA_TAMANO_Y],
  bool &flag_tabla_cargada, bool &flag_tabla_modificada,
  char direccion_archivo[RUTA_MAX]
) {
  int eleccion = -1;

  while (eleccion != 0) {
    // Con esta comparación se evita reimprimir el menú si
    // la opción ingresada es incorrecta, pero sí si se
    // acaba de salir de una opción real luego de utilizarla.
    if (eleccion < 8) {
      //cout << endl;
      cout << "Menú de opciones de control de memoria del dron." << endl;
      if (flag_tabla_cargada) {
        cout << "Archivo cargado: ";
        if (flag_tabla_modificada) cout << ANSI_ITALICS;
        cout << "\"" << direccion_archivo << "\"";
        // Se agrega un asterisco para denotar de manera visual que
        // se realizaron cambios a la tabla en memoria.
        if (flag_tabla_modificada) cout << ANSI_CLEAR << "*";
        cout << endl;
      }
      cout << "Por favor, selecciones una de las siguientes opciones: (ej: 1, 2, 3)" <<
endl;

      cout << "  [1]. Cargar archivo de ataque a memoria." << endl;

      if (!flag_tabla_cargada) cout << ANSI_ITALICS;
      // La idea es que en caso de no tener cargada en memoria un archivo
      // indicar visualmente que la opción no es accesible.
      cout << "  [2]. Mostrar archivo cargado." << ANSI_CLEAR << endl;

      cout << "  [3]. Crear un archivo de ataque nuevo." << endl;

      cout << "  [4]. Corregir un registro del archivo." << endl;

      if (!flag_tabla_cargada) cout << ANSI_ITALICS;
      cout << "  [5]. Corregir un registro en memoria." << ANSI_CLEAR << endl;

      if (!flag_tabla_cargada) cout << ANSI_ITALICS;
      cout << "  [6]. Guardar tabla en memoria en un archivo nuevo." << ANSI_CLEAR << endl;

      if (!flag_tabla_cargada) cout << ANSI_ITALICS;
      cout << "  [7]. Visualizar un archivo de ataque en HTML." << ANSI_CLEAR << endl;

      cout << "  [0]. Terminar este programa." << endl;
    }

    cout << CURSOR;
    cin >> eleccion;
    if (cinInvalido()) {
```

```
      eleccion = 8;
      continue;
    }
    // Se utiliza cin.ignore() para prevenir error de leading whitespace al presionar Enter.
    // Este comportamiento se puede obtener utilizando cin.get(), pero se decidió por
    // usar cin.ignore() ya que el valor obtenido no va a ser almacenado.
    cin.ignore();
    switch (eleccion) {
      default:
        cout << ERROR; // MSG_ERROR
        cout << "Opción no valida. Vuelva a intentarlo." << endl;
        break;
      case 0:
        cout << "Terminando ejecución." << endl;
        break;
      case 1: // Cargar archivo de ataque.
        cargarArchivo(tabla, direccion_archivo);
        flag_tabla_cargada = true;
        flag_tabla_modificada = false;
        break;
      case 2: // Mostrar archivo cargado.
        if (!comprobarTablaCargada(flag_tabla_cargada)) break;
        mostrarTabla(tabla);
        break;
      case 3: // Crear archivo de ataque nuevo y poblarlo.
        crearAtaque();
        break;
      case 4: // Modificar tabla en archivo.
        modificarRegistroDeArchivo();
        break;
      case 5: // Modificar tabla en memoria.
        if (!comprobarTablaCargada(flag_tabla_cargada)) break;
        flag_tabla_modificada = modificarRegistro(tabla);
        break;
      case 6: // Guardar archivo de ataque.
        if (!comprobarTablaCargada(flag_tabla_cargada)) break;
        guardarAtaque(tabla, direccion_archivo);
        flag_tabla_modificada = false;
        break;
      case 7: // Crear página HTML.
        if (!comprobarTablaCargada(flag_tabla_cargada)) break;
        crearHTML(tabla, direccion_archivo);
        break;
    }
  }

  return;
}
```

Se utilizan *secuencias ANSI* en múltiples puntos para comunicarle al emulador de terminal donde aplicar ciertos cambios estéticos, como `ANSI_ITALICS`, para denotar que cuando la *tabla* `tabla[TABLA_TAMANO_X][TABLA_TAMANO_Y]` está vacía, según el *flag* `flag_tabla_cargada`, dichas opciones no están disponibles para la ejecución.

```
C/C++
#define ANSI_ITALICS "\033[3m" // Código ANSI para Italics.
```

```
#define ANSI_CLEAR "\033[0m"
#define ANSI_LINEA_ANTERIOR "\033[F" // Mover el cursor de la terminal al inicio de la linea
anterior.
#define ERROR "\033[1mError\033[0m: " // "Error:" con el código ANSI de negrita.
#define WARN "\033[1mAlerta\033[0m: "
```

Luego de solicitar un valor numérico para seleccionar a una de las 7 opciones accesibles desde el menú principal, se accede al procedimiento `cinInvalido()`, que comprueba que el valor solicitado sea efectivamente uno numérico.

C/C++

```
    if (cinInvalido()) {
      eleccion = 8;
      continue;
    }

// Comprueba que se se haya ingresado un valor válido. En caso de que
// "cin" devuelva error, limpia el estado de error y devuelve "true".
bool cinInvalido() {
  if (!cin) {
    cout << ERROR; // MSG_ERROR
    cout << "Respuesta no valida. Vuelva a intentarlo." << endl;
    cin.clear();
    // Limpio el buffer de entrada completo, evitando que si se ingresaron
    // multiples caracteres, se ignoren todos y no solo el primero.
    cin.ignore(std::numeric_limits<std::streamsize>::max(), '\n');
    return true;
  }

  return false;
}
```

Este procedimiento valida el valor de `cin`, que retorna un error en caso de un valor inesperado en su ingreso, como puede ser un caracter cuando se solicita uno numérico. En caso de que se encuentre en un estado de error, se utiliza `cin.clear()` para reiniciar ese estado de error al normal, y se ignora la totalidad de los valores ingresados por `cin`, mostrando por pantalla un mensaje de error y finalmente retornando `true`, para indicarle al programa principal que se debe volver a solicitar un valor.

C/C++

```
      case 7: // Crear página HTML.
        if (!comprobarTablaCargada(flag_tabla_cargada)) break;
        crearHTML(tabla, direccion_archivo);
        break;
```

En las funciones del programa que requieren de que la *tabla* `tabla[TABLA_TAMANO_X][TABLA_TAMANO_Y]` tenga registros válidos en su interior, se accede a la función `comprobarTablaCargada()`.

```C/C++
// Imprime un error y retorna "false" cuando no hay una tabla de
// órdenes cargada en memoria. Retorna "true" en caso contrario.
bool comprobarTablaCargada(bool flag) {
  if (!flag) {
    cout << WARN; // MSG_WARN
    cout << "No hay una tabla de órdenes cargada." << endl;
    cout << "Cargue primero una tabla de órdenes a la memoria del programa para acceder a
esta función." << endl;
    return false;
  }

  return true;
}
```

Que valida el *flag* `flag_tabla_cargada` , y devuelve `false` en caso de que el *flag* sea `false`, junto a un mensaje de alerta por pantalla.

Cargar archivo

Como explico en el comentario, durante la totalidad del proyecto, se utiliza la sentencia `while ()` de manera inversa a la intencionada, sin un valor de salida de bucle, sinó que invierto la lógica, utilizando sentencias `if ()` para determinar puntos clave donde volver al inicio del bucle. Esto lo hago tanto por una mejora de legibilidad del código, como por una de granularidad de los mensajes de error.

En vez de llevar pendiente una variable que haga la de flag de salida del bucle, repito de manera instantánea el bucle utilizando la palabra clave `continue` cuando una verificación no se cumple, permitiendome también identificar con exactitud en que momento del proceso se detuvo, pudiendo mostrar por pantalla un error concreto; y saliendo del bucle llegado al final con la palabra clave `break`.

Esta manera inversa de utilizar la sentencia `while ()` me permite granularizar los mensajes de error y alerta, logrando que estos sean más concretos con las acciones inválidas que puedan estar ocurriendo de parte del usuario, en vez de simplemente anular las acciones del usuario y no dar explicaciones.

```
C/C++
void cargarArchivo(Orden tabla[TABLA_TAMANO_X][TABLA_TAMANO_Y], char
direccion_archivo[RUTA_MAX]){
  OrdenArchivo lista_archivo[TABLA_TAMANO_X * TABLA_TAMANO_Y];

  // Acá estoy usando un While de una manera particular. Generalmente se le pone una
  // condición de salida y se comprueba cada loop, utilizando sentencias
  // selectivas para lograr dicha condición de salida.
  // En este caso ocurre lo contrario, no hay condición de salida y las sentencias
  // selctivas son las que repiten el bucle, pero al llegar al final del bucle
  // se consigue salir por la sentencia Break. Se eligió esta forma por la sencillez y
  // claridad visual de su implementación, ya que permite secuenciar multiples
  // condiciones de salida en sentencias If de manera clara.
  cout << "Por favor, ingrese una ruta de un archivo a cargar." << endl;
  while (true) {
    if (!buscarArchivo(direccion_archivo, 'r')) continue; // Vuelve a empezar el loop while.
    if (!leerArchivo(direccion_archivo, lista_archivo)) continue;
    listaATabla(lista_archivo, tabla);
    if (!validarTabla(tabla)) continue;
    // El mensaje de "archivo guardardo" está acá y no en "leerArchivo()" porque
    // se debe mostrar posterior a la comprovación de validez.
    cout << "Archivo cargado correctamente." << endl;
    cout << "Presione Enter para continuar… "; // MSG_SUCCESS
    cin.ignore();
    break;
  }

  return;
}
```

Para cargar un archivo *tabla de órdenes* a la memoria, se llevan a cabo una serie de comprobaciones. En primer lugar, se accede al procedimiento `buscarArchivo(direccion_archivo, 'r')`:

C/C++

```
// Comprueba la existencia de un archivo en una dirección dada.
// Retorna "false" si no existe.
bool buscarArchivo(char direccion_archivo[RUTA_MAX], char modo) {
  FILE *archivo;
  int tamaño_archivo = 0;
  char respuesta = '\0';

  cout << CURSOR;
  cin.getline(direccion_archivo, RUTA_MAX);
  //cin >> direccion_archivo;

  switch (modo) {
    case 'r':
      if (!(archivo = fopen(direccion_archivo, "r"))) {
        cout << ERROR; // MSG_ERROR
        cout << "La dirección del archivo ingresada es inválida o el archivo no existe. ";
        cout << "Por favor, vuelva a intentarlo o ingrese una ruta válida." << endl;
        return false;
      }

      fseek(archivo, 0, SEEK_END);
      tamaño_archivo = ftell(archivo) / sizeof(OrdenArchivo); // Tamaño del archivo en
celdas.
      if (tamaño_archivo != TABLA_TAMANO_X * TABLA_TAMANO_Y) {
        cout << ERROR; // MSG_ERROR
        cout << "El archivo cargado tiene un tamaño invalido. ";
        cout << "Por favor, ingrese un archivo de ataque válido." << endl;
        fclose(archivo);
        return false;
      }
      fclose(archivo);
      break;
    case 'w':
      ⋮
      break;
  }

  return true;
}
```

Este procedimiento toma como parámetro `char direccion_archivo[RUTA_MAX]` y `char modo`, para almacenar la dirección de un archivo y el modo de interacción que se va a tener con este.

La función primero solicita la dirección de un archivo, en el caso de la función `cargarArchivo()`, es para la lectura de este, así que el parámetro modo es `'r'`.

C/C++

```
case 'r':
      if (!(archivo = fopen(direccion_archivo, "r"))) {
        cout << ERROR; // MSG_ERROR
        cout << "La dirección del archivo ingresada es inválida o el archivo no existe. ";
        cout << "Por favor, vuelva a intentarlo o ingrese una ruta válida." << endl;
```

```
      return false;
    }

    fseek(archivo, 0, SEEK_END);
  tamaño_archivo = ftell(archivo) / sizeof(OrdenArchivo); // Tamaño del archivo en celdas.
    if (tamaño_archivo != TABLA_TAMANO_X * TABLA_TAMANO_Y) {
      cout << ERROR; // MSG_ERROR
      cout << "El archivo cargado tiene un tamaño invalido. ";
      cout << "Por favor, ingrese un archivo de ataque válido." << endl;
      fclose(archivo);
      return false;
    }
    fclose(archivo);
    break;
```

En este punto la función intenta abrir la localización del archivo indicado; si no lo logra, osea que la dirección ingresada para el archivo no existe, la función retorna `false` junto a mostrar por pantalla un error.

Si el archivo existe, y se pudo leer de manera correcta, procede a validar que el tamaño del archivo sea el esperado, y nuevamente si no es así, la función retorna `false` junto a mostrar por pantalla un error.

Finalmente, si ambas comprobaciones son exitosas, osea que el archivo ingresado existe y tiene un tamaño válido, entonces la función retorna `true`.

Una vez validado que la dirección ingresada por el usuario corresponde con un archivo real con un tamaño válido, se continua con el procedimiento `leerArchivo(direccion_archivo, lista_archivo)`:

C/C++

```
// Lee a memoria un archivo dada una dirección.
bool leerArchivo(
  char direccion_archivo[RUTA_MAX],
  OrdenArchivo lista[TABLA_TAMANO_X * TABLA_TAMANO_Y]
) {
  FILE *archivo;

  cout << "Cargando archivo de ataque ";
  cout << "\"" << direccion_archivo << "\"";
  cout << "…" << endl;

  archivo = fopen(direccion_archivo, "r+b");
  if (!fread(lista, sizeof(OrdenArchivo), TABLA_TAMANO_X * TABLA_TAMANO_Y, archivo)) {
    cout << ERROR; // MSG_ERROR
    cout << "Error al leer ";
    cout << "\"" << direccion_archivo << "\"" << endl;
    cout << ". Puede que el archivo se haya movido o borrado. Vuelva a intentarlo." << endl;
    return false;
  }
  //cout << "Archivo cargado correctamente." << endl;
  //cout << "Presione Enter para continuar… "; // MSG_SUCCESS
  //cin.ignore();
  fclose(archivo);
```

```
  return true;
}
```

Esta función toma por parámetros una dirección de archivo `char direccion_archivo[RUTA_MAX]` y una lista `OrdenArchivo lista[TABLA_TAMANO_X * TABLA_TAMANO_Y]`, y sencillamente carga los contenidos del archivo `direccion_archivo` en la lista `lista`.

Seguidamente, la función `cargarArchivo` convierte el contenido de la lista `lista_archivo` con la función `listaATabla(lista_archivo, tabla)`.

Finalmente, se validan los *registros* de la tabla con la función `validarTabla(tabla)`.

Si uno solo de estos pasos falla, su respectiva sentencia `if ()` llevará a cabo la palabra clave `continue`, que volverá a iniciar todo el proceso desde el inicio. En caso contrario, se mostrará por pantalla un mensaje de éxito, y el contenido de `tabla` se habrá poblado con los del archivo indicado por el usuario. También se activará el *flag* `flag_tabla_cargada`, para permitirle al usuario el acceso a las funciones que requieran de una tabla en memoria, puesto que se acaba de cargar una a esta.

Mostrar tabla

```C/C++
// Muestra el contenido de una tabla de órdenes por pantalla.
void mostrarTabla(Orden tabla[TABLA_TAMANO_X][TABLA_TAMANO_Y]) {
  for (int y = 0; y < TABLA_TAMANO_Y; y++) {
    for (int x = 0; x < TABLA_TAMANO_X; x++) {
      mostrarRegistro(tabla[x][y], x, y);
    }
  }

  return;
}
```

La función `mostrarTabla(tabla)` muestra los contenidos de la tabla de órdenes cargada en memoria registro a registro por pantalla.

```C/C++
// Muestra el contenido de un registro por pantalla.
void mostrarRegistro(Orden registro, int x, int y) {
  int digitos = 0;

  // Un poquito de diseño de tui.
  cout << "┌──────────────────────────────────────┐" << endl;

  cout << "│ " << "Posición actual (X;Y): ";
  for (int i = x + 1; i != 0; i /= 10) digitos++;
  for (int i = y + 1; i != 0; i /= 10) digitos++;
  digitos += 3; // "(", ";" y ")".
  for (int j = 0; j < 14 - digitos - 1; j++) cout << " ";
  digitos = 0;
  cout << "(" << x + 1 << ";" << y + 1 << ")" << endl;
  // Se le suma 1 a las coordenadas para volverlas amistosas para el usuario.
  // (Se interpreta que la tabla comienza en 1x1 y no en 0x0).

  cout << "│ " << "Despegue:                    ";
  cout << (registro.despegue ? "Veradero" : "   Falso") << endl;
  // Operador ternario para legibilidad.

  cout << "│ " << "Aterrizaje:                  ";
  cout << (registro.aterrizaje ? "Veradero" : "   Falso") << endl;

  cout << "│ " << "Soltar granada nº1:          ";
  cout << (registro.soltar_granada1  ? "Veradero" : "   Falso") << endl;

  cout << "│ " << "Soltar granada nº2:          ";
  cout << (registro.soltar_granada2 ? "Veradero" : "   Falso") << endl;

  cout << "│ " << "Ataque kamikaze:             ";
  cout << (registro.ataque_kamikaze ? "Veradero" : "   Falso") << endl;

  cout << "│ " <<  "Espera [Segundos]: ";
  for (int i = registro.espera; i != 0; i /= 10) digitos++; // Para alinear el texto a la
derecha
  for (int j = 0; j < 16 - digitos - 1; j++) cout << " ";   // sin importar la cantidad de
tiempo.
```

```
  digitos = 0;
  cout << registro.espera << " s" << endl;

  cout << "│ " << "Posición siguiente (X;Y): ";
  if (registro.aterrizaje || registro.ataque_kamikaze) cout << "     Final";
  else {
    for (int i = registro.siguiente_x + 1; i != 0; i /= 10) digitos++;
    for (int i = registro.siguiente_y + 1; i != 0; i /= 10) digitos++;
    digitos += 3; // "(", ";" y ")".
    for (int j = 0; j < 11 - digitos - 1; j++) cout << " ";
    digitos = 0;
    cout << "(" << registro.siguiente_x + 1 << ";" << registro.siguiente_y + 1 << ")";
  }
  cout << endl;

  cout << "└─ Presione Enter para continuar… ──────┘ ";
  cin.ignore(); // Presione Enter para continuar.
  cout << ANSI_LINEA_ANTERIOR;
  cout << "└─────────────────────────────────────┘ " << endl;

  return;
}
```

Muestra un registro dado como parámetro, variable por variable, en un formato estético visualmente.

```
None
┌─────────────────────────────────────┐
 Posición actual (X;Y):         (4;2)
 Despegue:                  Verdadero
 Aterrizaje:                    Falso
 Soltar granada nº1:            Falso
 Soltar granada nº2:            Falso
 Ataque kamikaze:               Falso
 Espera [Segundos]:              25 s
 Posición siguiente (X;Y):      (4;1)
└─ Presione Enter para continuar… ──┘
```

También utilizo un bucle `for ()` para identificar cuantos digitos tienen los números mostrados por pantalla, para así poder determinar cuántos espacios en blanco se deben desplazar a la derecha para que todos los valores estén justificados del mismo modo.

```
C/C++
  cout << "│ " <<  "Espera [Segundos]: ";
  for (int i = registro.espera; i != 0; i /= 10) digitos++; // Para alinear el texto a la
derecha
  for (int j = 0; j < 16 - digitos - 1; j++) cout << " ";   // sin importar la cantidad de
tiempo.
  digitos = 0;
  cout << registro.espera << " s" << endl;
```

Finalmente, se le solicita al usuario interactuar con el programa para continuar al registro siguiente, presionando la tecla *Enter*.

```C/C++
  cout << "└─ Presione Enter para continuar… ─────┘ ";
  cin.ignore(); // Presione Enter para continuar.
  cout << ANSI_LINEA_ANTERIOR;
  cout << "└──────────────────────────────────────┘ " << endl;
```

Acá se utiliza una secuencia *ANSI* para, al usuario presionar el *Enter*, volver a la línea anterior y sobreescribirla con una línea continua, sobreescribiendo el mensaje de `Presione Enter para continuar…` .

Una vez recorrida la tabla por completo, se vuelve al menú principal.

Crear ataque

```C/C++
// Solicita valores al usuario para crear una tabla que luego almacena en
// un archivo nuevo.
void crearAtaque() {
  // "crearAtaque()" no pide como parametro la tabla de órdenes de la memoria,
  // ya que esta no se utiliza fuera de su propio procedimiento.
  // Está contenida dentro de la creación del archivo en donde se almacenará.
  // Para interactuar con el resto de funciones, como "mostrarTabla()",
  // por ejemplo, se debe cargar como cualquier otro archivo.
  char direccion_archivo[RUTA_MAX] = { '\0' };
  OrdenArchivo lista_archivo[TABLA_TAMANO_X * TABLA_TAMANO_Y];
  Orden tabla[TABLA_TAMANO_X][TABLA_TAMANO_Y];

  poblarTabla(tabla);
  tablaALista(lista_archivo, tabla);

  cout << "Por favor, ingrese la ruta donde guardar el archivo." << endl;
  while (true) {
    if (!buscarArchivo(direccion_archivo, 'w')) continue;
    if (!escribirArchivo(direccion_archivo, lista_archivo)) continue;
    break;
  }

  return;
}
```

Para crear un archivo de tabla de órdenes, primero se crea una tabla `Orden tabla[TABLA_TAMANO_X][TABLA_TAMANO_Y]`, que se procede a poblar con la función `poblarTabla(tabla)`:

```C/C++
// Solicita valores al usuario para poblar una tabla, registro a registro.
void poblarTabla(Orden tabla[TABLA_TAMANO_X][TABLA_TAMANO_Y]) {
  Flags flags;
  int x = -1, y = -1, siguiente_x = -1, siguiente_y = -1;

  while (true) {
    while (true) { // Qué registro.
      cout << "Introduzca celda de despegue." << endl;
      cout << CURSOR << "X: ";
      cin >> x;
      if (cinInvalido()) continue;
      cout << CURSOR << "Y: ";
      cin >> y;
      if (cinInvalido()) continue;
      x--; // (Se interpreta que la tabla comienza en 1x1 y no en 0x0,
      y--; // así que se decrementan ambos).

      if (
        x >= TABLA_TAMANO_X ||
        y >= TABLA_TAMANO_Y
      ) {
        cout << ERROR; // MSG_ERROR
```

```
        cout << "Celda fuera de límites. Elija otra posición." << endl;
        continue;
      }
      cin.ignore(); // Leading whitespace.
      break;
    }
    tabla[x][y].despegue = true;
    flags.despegue = true;
    flags.posiciones_transitadas[x][y] = true;
    while (true) {
      poblarRegistro(tabla[x][y], x, y, flags);
      if (flags.aterrizaje || flags.kamikaze) break;
      siguiente_x = tabla[x][y].siguiente_x;
      siguiente_y = tabla[x][y].siguiente_y;
      x = siguiente_x;
      y = siguiente_y;
    }

    if (!validarTabla(tabla)) {
      cout << ERROR; // MSG_ERROR
      cout << "La tabla de órdenes creada es inválida. Vuelva a crearla." << endl;
      // Este mensaje después de introducir 40 mil registros
      // debe ser horrible, por decirlo de manera no bulgar.
      continue;
    }
    break;
  }

  return;
}
```

Se solicita una celda de despegue, y una vez validada la celda, se asigna de despegue y se procede a poblar el resto de la tabla con `poblarRegistro(tabla[x][y], x, y, flags)`.

C/C++

```
void poblarRegistro(Orden &registro, int x, int y, Flags &flags) {
  char respuesta = '\0';
  unsigned int respuesta_tiempo = 0;
  unsigned int respuesta_x = 0, respuesta_y = 0;

  cout << "Posición (" << x + 1 << ";" << y + 1 << "):" << endl;

  while (!flags.despegue) { // Despegue.
    cout << "  ¿Despegue? [s/N]" << endl;
    cout << "  " << CURSOR;
    cin.get(respuesta);

    switch (respuesta) {
      default:
        cout << ERROR; // MSG_ERROR
        cout << "Respuesta no valida. Vuelva a intentarlo." << endl;
        cin.ignore(std::numeric_limits<std::streamsize>::max(), '\n'); // Leading
whitespace.
        continue;
      case 's':
```

```
      case 'S':
        registro.despegue = true;
        registro.aterrizaje = false;
        registro.ataque_kamikaze = false;
        registro.soltar_granada1 = false;
        registro.soltar_granada2 = false;
        flags.despegue = true;
        flags.posiciones_transitadas[x][y] = true;
        break;
      case 'n':
      case 'N':
      case '\n':
        registro.despegue = false;
        break;
    }
    if (respuesta != '\n') cin.ignore(); // Leading whitespace.
    break;
  }

  // Aterrizaje.
  while (
    !flags.aterrizaje &&
    !flags.kamikaze   &&
    !registro.despegue
  ) {
    cout << "  ¿Aterrizaje? [s/N]" << endl;
    cout << "  " << CURSOR;
    cin.get(respuesta);

    switch (respuesta) {
      default:
        cout << ERROR; // MSG_ERROR
        cout << "Respuesta no valida. Vuelva a intentarlo." << endl;
        cin.ignore(std::numeric_limits<std::streamsize>::max(), '\n'); // Leading
whitespace.
        continue;
      case 's':
      case 'S':
        registro.despegue = false;
        registro.aterrizaje = true;
        registro.ataque_kamikaze = false;
        flags.aterrizaje = true;
        return;
      case 'n':
      case 'N':
      case '\n':
        registro.aterrizaje = false;
        break;
    }
    if (respuesta != '\n') cin.ignore(); // Leading whitespace.
    break;
  }

  // Kamikaze.
  while (
    !flags.aterrizaje &&
    !flags.kamikaze   &&
    !registro.despegue
  ) {
    cout << "  ¿Ataque kamikaze? [s/N]" << endl;
```

```
    cout << "  " << CURSOR;
    cin.get(respuesta);

    switch (respuesta) {
      default:
        cout << ERROR; // MSG_ERROR
        cout << "Respuesta no valida. Vuelva a intentarlo." << endl;
        cin.ignore(std::numeric_limits<std::streamsize>::max(), '\n'); // Leading
whitespace.
        continue;
      case 's':
      case 'S':
        registro.despegue = false;
        registro.aterrizaje = false;
        registro.ataque_kamikaze = true;
        flags.kamikaze = true;
        return;
      case 'n':
      case 'N':
      case '\n':
        registro.ataque_kamikaze = false;
        break;
    }
    if (respuesta != '\n') cin.ignore(); // Leading whitespace.
    break;
  }

  // Granada nº1.
  while (
    !flags.granada1 &&
    !registro.despegue
  ) {
    cout << "  ¿Soltar granada nº1? [s/N]" << endl;
    cout << "  " << CURSOR;
    cin.get(respuesta);

    switch (respuesta) {
      default:
        cout << ERROR; // MSG_ERROR
        cout << "Respuesta no valida. Vuelva a intentarlo." << endl;
        cin.ignore(std::numeric_limits<std::streamsize>::max(), '\n'); // Leading
whitespace.
        continue;
      case 's':
      case 'S':
        registro.despegue = false;
        registro.soltar_granada1 = true;
        flags.granada1 = true;
        break;
      case 'n':
      case 'N':
      case '\n':
        registro.soltar_granada1 = false;
        break;
    }
    if (respuesta != '\n') cin.ignore(); // Leading whitespace.
    break;
  }

  // Granada nº2.
```

```
  while (
    !flags.granada2 &&
    !registro.despegue
  ) {
    cout << "  ¿Soltar granada nº2? [s/N]" << endl;
    cout << "  " << CURSOR;
    cin.get(respuesta);

    switch (respuesta) {
      default:
        cout << ERROR; // MSG_ERROR
        cout << "Respuesta no valida. Vuelva a intentarlo." << endl;
        cin.ignore(std::numeric_limits<std::streamsize>::max(), '\n'); // Leading
whitespace.
        continue;
      case 's':
      case 'S':
        registro.despegue = false;
        registro.soltar_granada2 = true;
        flags.granada2 = true;
        break;
      case 'n':
      case 'N':
      case '\n':
        registro.soltar_granada2 = false;
        break;
    }
    if (respuesta != '\n') cin.ignore(); // Leading whitespace.
    break;
  }

  // Tiempo de espera.
  while (true) {
    cout << "  Tiempo de espera (Segundos)." << endl;
    cout << "  " << CURSOR;
    cin >> respuesta_tiempo;
    if (cinInvalido()) continue;
    if (respuesta_tiempo == 0) {
      cout << ERROR; // MSG_ERROR
      cout << "Ingrese un valor de tiempo de espera diferente a 0." << endl;
      continue;
    }
    registro.espera = respuesta_tiempo;
    break;
  }

  // Siguiente posición.
  while (
    !registro.aterrizaje &&
    !registro.ataque_kamikaze
  ) {
    cout << "  Posición siguiente (X;Y)." << endl;
    cout << "  " << CURSOR << "X: ";
    cin >> respuesta_x;
    if (cinInvalido()) continue;
    cout << "  " << CURSOR << "Y: ";
    cin >> respuesta_y;
    if (cinInvalido()) continue;
    respuesta_x--;
    respuesta_y--;
```

```
    // (Se interpreta que la tabla comienza en 1x1 y no en 0x0).

    if (
      !(respuesta_x == x     && respuesta_y == y - 1) && // Norte.
      !(respuesta_x == x + 1 && respuesta_y == y)     && // Este.
      !(respuesta_x == x     && respuesta_y == y + 1) && // Sur.
      !(respuesta_x == x - 1 && respuesta_y == y)        // Oeste.
    ) {
      cout << ERROR; // MSG_ERROR
      cout << "Seleccione una celda adyasente." << endl;
      continue;
    }

    if (
      respuesta_x >= TABLA_TAMANO_X ||
      respuesta_y >= TABLA_TAMANO_Y
    ) {
      cout << ERROR; // MSG_ERROR
      cout << "Celda fuera de límites. Elija otra posición." << endl;
      continue;
    }

    if (flags.posiciones_transitadas[respuesta_x][respuesta_y]) {
      cout << ERROR; // MSG_ERROR
      cout << "Celda ya transitada. Elija otra posición." << endl;
      continue;
    }

    registro.siguiente_x = respuesta_x;
    registro.siguiente_y = respuesta_y;
    flags.posiciones_transitadas[respuesta_x][respuesta_y] = true;
    break;
  }

  cin.ignore(); // Leading whitespace.

  return;
}
```

None

```
  cout << "└─ Presione Enter para continuar… ──────┘ ";
  cin.ignore(); // Presione Enter para continuar.
  cout << ANSI_LINEA_ANTERIOR;
  cout << "└──────────────────────────────────────────┘ " << endl;
```

Modificar registro

C/C++

```
// Solicita uno o más registros de una tabla dada para modificar.
bool modificarRegistro(Orden tabla[TABLA_TAMANO_X][TABLA_TAMANO_Y]) {
  char respuesta = '\0';
  unsigned int respuesta_x = 0, respuesta_y = 0;
  Orden tabla_nueva[TABLA_TAMANO_X][TABLA_TAMANO_Y];
  Flags *flags;

  // Para poder anular los cambios en caso de que sean inválidos.
  clonarTabla(tabla, tabla_nueva);

  do {
    while (true) { // Qué registro.
      cout << "¿Qué registro desea modificar? (X;Y)" << endl;
      cout << CURSOR << "X: ";
      cin >> respuesta_x;
      if (cinInvalido()) continue;
      cout << CURSOR << "Y: ";
      cin >> respuesta_y;
      if (cinInvalido()) continue;
      respuesta_x--; // (Se interpreta que la tabla comienza en 1x1 y no en 0x0,
      respuesta_y--; // así que se decrementan ambos).

      if (
        respuesta_x >= TABLA_TAMANO_X ||
        respuesta_y >= TABLA_TAMANO_Y
      ) {
        cout << ERROR; // MSG_ERROR
        cout << "Celda fuera de límites. Elija otra posición." << endl;
        continue;
      }
      cin.ignore(); // Leading whitespace.

      break;
    }

    cout << "Registro original:" << endl;
    mostrarRegistro(tabla_nueva[respuesta_x][respuesta_y], respuesta_x, respuesta_y);

    flags = new Flags();
    // ~He construído un castillo desde afuera hacia adentro, y ahora que yazco en la última
de las cámaras,
    // sin puertas ni ventanas, solo me resta una única salida: cavar un agujero~
    poblarRegistro(tabla_nueva[respuesta_x][respuesta_y], respuesta_x, respuesta_y, *flags);
    delete flags;

    cout << endl << "Resultado:" << endl;
    mostrarRegistro(tabla_nueva[respuesta_x][respuesta_y], respuesta_x, respuesta_y);

    while (true) {
      cout << "¿Cambiar otro registro? [s/N]" << endl;
      cout << CURSOR;
      cin.get(respuesta);
      switch (respuesta) {
        default:
          cout << ERROR; // MSG_ERROR
          cout << "Respuesta no valida. Vuelva a intentarlo." << endl;
```

```
          cin.ignore(std::numeric_limits<std::streamsize>::max(), '\n'); // Leading
whitespace.
          continue;
        case 's':
        case 'S':
        case 'n':
        case 'N':
        case '\n':
          break;
      }
      if (respuesta != '\n') cin.ignore(); // Leading whitespace.
      break;
    }
  } while (respuesta == 's' || respuesta == 'S');

  while (true) { // Guardar cambios.
    cout << "¿Guardar cambios? [S/n]" << endl;
    cout << CURSOR;
    cin.get(respuesta);

    switch (respuesta) {
      default:
        cout << ERROR; // MSG_ERROR
        cout << "Respuesta no valida. Vuelva a intentarlo." << endl;
        cin.ignore(); // Leading whitespace.
        continue;
      case 's':
      case 'S':
      case '\n':
        cout << "Validando cambios…" << endl;
        if (!validarTabla(tabla_nueva)) {
          cout << ERROR; // MSG_ERROR
          cout << "Las modificaciones hechas a la tabla son inválidas.";
          cout << " Deshaciendo los cambios." << endl;
          clonarTabla(tabla, tabla_nueva);
          return false;
        }
        cout << "Cambios validados. Todo en orden." << endl;
        clonarTabla(tabla_nueva, tabla);
        cout << "Presione Enter para continuar… "; // MSG_SUCCESS
        cin.ignore();
        return true;
      case 'n':
      case 'N':
        clonarTabla(tabla, tabla_nueva);
        cout << "Se han deshecho los cambios." << endl;
        return false;
    }
  }
}
```

Para modificar un registro individual de una tabla cargada en memoria, primero se copia el contenido de la tabla a modificar en una tabla secundaria donde se efectuarán los cambios antes de validarlos.

```C/C++
  // Para poder anular los cambios en caso de que sean inválidos.
  clonarTabla(tabla, tabla_nueva);
```

Entonces, el programa procederá a solicitarle al usuario a cual celda acceder para modificar sus contenidos.

```C/C++
    while (true) { // Qué registro.
      cout << "¿Qué registro desea modificar? (X;Y)" << endl;
      cout << CURSOR << "X: ";
      cin >> respuesta_x;
      if (cinInvalido()) continue;
      cout << CURSOR << "Y: ";
      cin >> respuesta_y;
      if (cinInvalido()) continue;
      respuesta_x--; // (Se interpreta que la tabla comienza en 1x1 y no en 0x0,
      respuesta_y--; // así que se decrementan ambos).

      if (
        respuesta_x >= TABLA_TAMANO_X ||
        respuesta_y >= TABLA_TAMANO_Y
      ) {
        cout << ERROR; // MSG_ERROR
        cout << "Celda fuera de límites. Elija otra posición." << endl;
        continue;
      }
      cin.ignore(); // Leading whitespace.

      break;
    }
```

El programa mostrará por pantalla el contenido original de la celda, y solicitará uno por uno los valores del registro nuevo.

```C/C++
    cout << "Registro original:" << endl;
    mostrarRegistro(tabla_nueva[respuesta_x][respuesta_y], respuesta_x, respuesta_y);

    flags = new Flags();
    // ~He construído un castillo desde afuera hacia adentro, y ahora que yazco en la última
de las cámaras,
    // sin puertas ni ventanas, solo me resta una única salida: cavar un agujero~
    poblarRegistro(tabla_nueva[respuesta_x][respuesta_y], respuesta_x, respuesta_y, *flags);
    delete flags;

    cout << endl << "Resultado:" << endl;
    mostrarRegistro(tabla_nueva[respuesta_x][respuesta_y], respuesta_x, respuesta_y);
```

Una vez ingresados los valores nuevos, el programa verificará la validez de la tabla con los cambios implementados, y en caso de no ser válidos, desechará los cambios efectuados.

```C/C++
  while (true) { // Guardar cambios.
    cout << "¿Guardar cambios? [S/n]" << endl;
    cout << CURSOR;
    cin.get(respuesta);

    switch (respuesta) {
      default:
        cout << ERROR; // MSG_ERROR
        cout << "Respuesta no valida. Vuelva a intentarlo." << endl;
        cin.ignore(); // Leading whitespace.
        continue;
      case 's':
      case 'S':
      case '\n':
        cout << "Validando cambios…" << endl;
        if (!validarTabla(tabla_nueva)) {
          cout << ERROR; // MSG_ERROR
          cout << "Las modificaciones hechas a la tabla son inválidas.";
          cout << " Deshaciendo los cambios." << endl;
          clonarTabla(tabla, tabla_nueva);
          return false;
        }
        cout << "Cambios validados. Todo en orden." << endl;
        clonarTabla(tabla_nueva, tabla);
        cout << "Presione Enter para continuar… "; // MSG_SUCCESS
        cin.ignore();
        return true;
      case 'n':
      case 'N':
        clonarTabla(tabla, tabla_nueva);
        cout << "Se han deshecho los cambios." << endl;
        return false;
    }
  }
```

Si los cambios hechos son válidos, se transferirán los contenidos modificados de la tabla modificada a la tabla original en memoria, y el programa volverá al menú principal.

Modificar registro de archivo

```C/C++
// Solicita uno o más registros de una tabla solicitada de un archivo para modificar.
void modificarRegistroDeArchivo() {
  char direccion_archivo[RUTA_MAX] = { '\0' };
  OrdenArchivo lista_archivo[TABLA_TAMANO_X * TABLA_TAMANO_Y];
  Orden tabla[TABLA_TAMANO_X][TABLA_TAMANO_Y];

  cout << "Por favor, ingrese una ruta de un archivo a cargar." << endl;
  while (true) {
    if (!buscarArchivo(direccion_archivo, 'r')) continue;
    if (!leerArchivo(direccion_archivo, lista_archivo)) continue;
    listaATabla(lista_archivo, tabla);
    if (!validarTabla(tabla)) continue;
    if (!modificarRegistro(tabla)) return;
    tablaALista(lista_archivo, tabla);
    if (!escribirArchivo(direccion_archivo, lista_archivo)) continue;
    break;
  }
  // Archivo -> Memoria (Lista) -> Memoria (Tabla) -> Se edita -> Memoria (Tabla) -> Memoria
(Lista) -> Archivo.
  // Efectivamente ineficiente.

  return;
}
```

El procedimiento para modificar un registro de un archivo es similar al de simplemente modificar uno de una tabla cargada en memoria, solo que conlleva más comprobaciones.

```C/C++
    if (!buscarArchivo(direccion_archivo, 'r')) continue;
    if (!leerArchivo(direccion_archivo, lista_archivo)) continue;
```

Primero se solicita la dirección del archivo a modificar con `buscarArchivo()`, y se carga el contenido del mismo archivo a la lista `lista_archivo` mediante `leerArchivo()`.

```C/C++
    listaATabla(lista_archivo, tabla);
    if (!validarTabla(tabla)) continue;
    if (!modificarRegistro(tabla)) return;
```

La lista se adapta a una tabla con `listaATabla()`, y se valida mediante `validarTabla()`.

```C/C++
    tablaALista(lista_archivo, tabla);
    if (!escribirArchivo(direccion_archivo, lista_archivo)) continue;
```

Se solicitan y modifican los registros indicados por el usuario con `modificarRegistro()`, y una vez finalizado el proceso, se vuelve a adaptar la tabla modificada a una lista y se almacena en el mismo archivo ingresado.

Si uno solo de estos procesos falla, el programa devuelve un error por pantalla y el bucle `while ()` vuelve a comenzar.

Guardar ataque

```C/C++
// Almacena la tabla de memoria que luego almacena en un archivo nuevo.
void guardarAtaque(Orden tabla[TABLA_TAMANO_X][TABLA_TAMANO_Y], char
direccion_archivo[RUTA_MAX]) {
  OrdenArchivo lista_archivo[TABLA_TAMANO_X * TABLA_TAMANO_Y];

  cout << "Por favor, ingrese la ruta donde guardar el archivo." << endl;
  while(true) {
    if (!buscarArchivo(direccion_archivo, 'w')) continue;
    tablaALista(lista_archivo, tabla);
    if (!escribirArchivo(direccion_archivo, lista_archivo)) continue;
    break;
  }

  return;
}
```

Para almacenar una tabla de órdenes de la memoria del programa en un archivo, primero el programa le solicita al usuario una dirección de archivo válida para escritura, utilizando `buscarArchivo()`, pasando como parámetro el modo `'w'`.
Una vez creado el archivo, se pasa el contenido de la tabla a una lista `lista_archivo` y se almacena en el archivo creado, con `escribirArchivo()`.

```C/C++
    if (!buscarArchivo(direccion_archivo, 'w')) continue;
    tablaALista(lista_archivo, tabla);
    if (!escribirArchivo(direccion_archivo, lista_archivo)) continue;
```

Crear HTML

C/C++

```
void crearHTML(
  Orden tabla[TABLA_TAMANO_X][TABLA_TAMANO_Y],
  char direccion_archivo[RUTA_MAX]
) {
  const char *nombre_archivo[RUTA_MAX] = { "\0" };
  char direccion_pagina[RUTA_MAX] = { '\0' };
  Orden registro, registro_siguiente;
  FILE *archivo_pagina;
  Flags flags;
  unsigned int x = 0, y = 0;
  bool norte = false, sur = false, este = false, oeste = false;
  const char *char_registro[TABLA_TAMANO_X][TABLA_TAMANO_Y] = { "\0" };

  *nombre_archivo = strrchr(direccion_archivo, '/'); // Si no hay un '/' en la dirección
  if (nombre_archivo[0] == NULL) *nombre_archivo = direccion_archivo; // se asigna completa.
  else nombre_archivo[0] = *nombre_archivo + 1;
  strcpy(direccion_pagina, direccion_archivo); // STRING
  strcat(direccion_pagina, ".html"); // En un mundo ideal donde todos usamos strings. //
STRING

  indexarTabla(tabla, flags); // Obtener "despegue_x" y "despegue_y".
  x = flags.despegue_x;
  y = flags.despegue_y;
  archivo_pagina = fopen(direccion_pagina, "w");

  while (true) {
    registro = tabla[x][y];
    registro_siguiente = tabla[registro.siguiente_x][registro.siguiente_y];
    if (registro.aterrizaje || registro.ataque_kamikaze) registro_siguiente = registro;

    if (
      registro.siguiente_x == x                                  &&
      registro.siguiente_y == y - 1                              ||
      registro_siguiente.siguiente_x == registro.siguiente_x     &&
      registro_siguiente.siguiente_y == registro.siguiente_y + 1
    ) {
      sur = true;
    }
    if (
      registro.siguiente_x == x + 1                               &&
      registro.siguiente_y == y                                   ||
      registro_siguiente.siguiente_x == registro.siguiente_x - 1  &&
      registro_siguiente.siguiente_y == registro.siguiente_y
    ) {
      oeste = true;
    }
    if (
      registro.siguiente_x == x                                   &&
      registro.siguiente_y == y + 1                               ||
      registro_siguiente.siguiente_x == registro.siguiente_x      &&
      registro_siguiente.siguiente_y == registro.siguiente_y - 1
    ) {
      norte = true;
    }
    if (
      registro.siguiente_x == x - 1                               &&
```

```
      registro.siguiente_y ==                                    ||
      registro_siguiente.siguiente_x == registro.siguiente_x + 1  &&
      registro_siguiente.siguiente_y == registro.siguiente_y
    ) {
      este = true;
    }

    if  (norte && sur)   char_registro[registro.siguiente_x][registro.siguiente_y] = "│";
    if  (sur && este)    char_registro[registro.siguiente_x][registro.siguiente_y] = " ┌─";
    if  (norte && este)  char_registro[registro.siguiente_x][registro.siguiente_y] = " └─";
    if  (sur && oeste)   char_registro[registro.siguiente_x][registro.siguiente_y] = "─┐ ";
    if  (norte && oeste) char_registro[registro.siguiente_x][registro.siguiente_y] = "─┘ ";
    if  (oeste && este)  char_registro[registro.siguiente_x][registro.siguiente_y] = "───";

    if      (registro.despegue)         char_registro[x][y] = "•";
    else if (registro.aterrizaje)       char_registro[x][y] = " ";
    else if (registro.ataque_kamikaze)  char_registro[x][y] = " ";
    else if (registro.soltar_granada1)  char_registro[x][y] = " ";
    else if (registro.soltar_granada2)  char_registro[x][y] = " ";

    if (registro.aterrizaje || registro.ataque_kamikaze) break; // Final del recorrido.

    norte  = false;
    sur    = false;
    este   = false;
    oeste  = false;
    x = registro.siguiente_x;
    y = registro.siguiente_y;
  }

  fprintf(archivo_pagina, "<html>\n\t<head>\n");
  fprintf(archivo_pagina, "\t<title>%s - TP Dron</title>", *nombre_archivo);
  fprintf(archivo_pagina, R"(
  </head>
  <body>
  <header style="padding: 5px; width: 100%;">
    <h1>Vista previa de la tabla de órdenes del dron.</h1>)");
  fprintf(archivo_pagina, "\n\t\t<h2>Tabla \"%s\"</h2>", *nombre_archivo);
  fprintf(archivo_pagina, R"(
    <hr width="100%"/>
  </header>
  <section style="display: flex; gap: 10px;">
    <div class="borde">
    <div class="scrolleable">
    <table class="table">
      <thead>
      <th><div class="content"></div></th>)");
  for (int x = 0; x < TABLA_TAMANO_X; x++) {
    fprintf(archivo_pagina, "\n\t\t\t<th><div class=\"content\">%d</div></th>", x + 1); //
Columna
  }
  fprintf(archivo_pagina, "\n\t\t\t</thead>");
  for (int k = 0; k < TABLA_TAMANO_Y; k++) {
    fprintf(archivo_pagina, "\n\t\t\t<tr>\n\t\t\t<td><div class=\"content\">%d</div></td>",
k + 1); // Fila.
    for (int j = 0; j < TABLA_TAMANO_X; j++) {
      fprintf(archivo_pagina, "\n\t\t\t<td title=\"Celda %d×%d", j + 1, k + 1); // Numero de
celda.
      if (!tabla[j][k].aterrizaje && !tabla[j][k].ataque_kamikaze) {
        fprintf(archivo_pagina, "&#10;Siguiente %d×%d",
```

```
          tabla[j][k].siguiente_x + 1, tabla[j][k].siguiente_y + 1
        );
      }
      if (tabla[j][k].despegue)         fprintf(archivo_pagina, "&#10;Celda de despegue");
      if (tabla[j][k].aterrizaje)       fprintf(archivo_pagina, "&#10;Celda de aterrizaje");
      if (tabla[j][k].ataque_kamikaze)  fprintf(archivo_pagina, "&#10;Ataque kamikaze");
      if (tabla[j][k].soltar_granada1)  fprintf(archivo_pagina, "&#10;Ataque granada nº1");
      if (tabla[j][k].soltar_granada2)  fprintf(archivo_pagina, "&#10;Ataque granada nº2");
      fprintf(archivo_pagina, "&#10;Tiempo de espera: %d s\"><div
class=\"content\">%s</div></td>",
        tabla[j][k].espera,
        char_registro[j][k]
      );
    }
    fprintf(archivo_pagina, "\n\t\t\t</tr>");
  }
  fprintf(archivo_pagina, R"(
    </table>

</html>)");

  fclose(archivo_pagina);
  cout << "El archivo ";
  cout << "\"" << direccion_pagina << "\"";
  cout << " se guardó correctamente." << endl;
  cout << "Presione Enter para continuar… "; // MSG_SUCCESS
  cin.ignore();

  return;
}
```

Primero, se obtiene la dirección del archivo cargado en memoria, y se le concatena el *string* `'.html'`, para asegurar que al momento de guardar la página HTML, el archivo pueda ser reconocido fácilmente como tal.

```C/C++
*nombre_archivo = strrchr(direccion_archivo, '/'); // Si no hay un '/' en la dirección
if (nombre_archivo[0] == NULL) *nombre_archivo = direccion_archivo; // se asigna completa.
else nombre_archivo[0] = *nombre_archivo + 1;
strcpy(direccion_pagina, direccion_archivo); // STRING
strcat(direccion_pagina, ".html"); // En un mundo ideal donde todos usamos strings. //
STRING
```

Esto se hace mediante el uso de la librería `<string.h>`. Este es el único uso de la librería en el programa.

Se continúa obteniendo la celda de despegue de la tabla con la función `indexarTabla(tabla, flags)`, y se determina el contenido de cada celda.

C/C++

```
while (true) {
  registro = tabla[x][y];
  registro_siguiente = tabla[registro.siguiente_x][registro.siguiente_y];
  if (registro.aterrizaje || registro.ataque_kamikaze) registro_siguiente = registro;

  if (
    registro.siguiente_x == x                                   &&
    registro.siguiente_y == y - 1                               ||
    registro_siguiente.siguiente_x == registro.siguiente_x      &&
    registro_siguiente.siguiente_y == registro.siguiente_y + 1
  ) {
    sur = true;
  }
  if (
    registro.siguiente_x == x + 1                               &&
    registro.siguiente_y == y                                   ||
    registro_siguiente.siguiente_x == registro.siguiente_x - 1  &&
    registro_siguiente.siguiente_y == registro.siguiente_y
  ) {
    oeste = true;
  }
  if (
    registro.siguiente_x == x                                   &&
    registro.siguiente_y == y + 1                               ||
    registro_siguiente.siguiente_x == registro.siguiente_x      &&
    registro_siguiente.siguiente_y == registro.siguiente_y - 1
  ) {
    norte = true;
  }
  if (
    registro.siguiente_x == x - 1                               &&
    registro.siguiente_y == y                                   ||
    registro_siguiente.siguiente_x == registro.siguiente_x + 1  &&
    registro_siguiente.siguiente_y == registro.siguiente_y
  ) {
    este = true;
  }

  if (norte && sur)    char_registro[registro.siguiente_x][registro.siguiente_y] = "│";
  if (sur && este)     char_registro[registro.siguiente_x][registro.siguiente_y] = " ┌";
  if (norte && este)   char_registro[registro.siguiente_x][registro.siguiente_y] = " └";
  if (sur && oeste)    char_registro[registro.siguiente_x][registro.siguiente_y] = "┐ ";
  if (norte && oeste)  char_registro[registro.siguiente_x][registro.siguiente_y] = "┘ ";
  if (oeste && este)   char_registro[registro.siguiente_x][registro.siguiente_y] = "──";

  if      (registro.despegue)         char_registro[x][y] = "•";
  else if (registro.aterrizaje)       char_registro[x][y] = " ";
  else if (registro.ataque_kamikaze)  char_registro[x][y] = " ";
  else if (registro.soltar_granada1)  char_registro[x][y] = " ";
  else if (registro.soltar_granada2)  char_registro[x][y] = " ";

  if (registro.aterrizaje || registro.ataque_kamikaze) break; // Final del recorrido.

  norte  = false;
  sur    = false;
  este   = false;
  oeste  = false;
  x = registro.siguiente_x;
  y = registro.siguiente_y;
}
```

Una vez determinado el contenido de cada celda, simplemente se transfieren una por una al archivo HTML.

```
C/C++
fprintf(archivo_pagina, "\n\t\t\t</thead>");
for (int k = 0; k < TABLA_TAMANO_Y; k++) {
  fprintf(archivo_pagina, "\n\t\t\t<tr>\n\t\t\t<td><div class=\"content\">%d</div></td>", k
+ 1); // Fila.
  for (int j = 0; j < TABLA_TAMANO_X; j++) {
    fprintf(archivo_pagina, "\n\t\t\t<td title=\"Celda %d×%d", j + 1, k + 1); // Numero de
celda.
    if (!tabla[j][k].aterrizaje && !tabla[j][k].ataque_kamikaze) {
      fprintf(archivo_pagina, "&#10;Siguiente %d×%d",
        tabla[j][k].siguiente_x + 1, tabla[j][k].siguiente_y + 1
      );
    }
    if (tabla[j][k].despegue)         fprintf(archivo_pagina, "&#10;Celda de despegue");
    if (tabla[j][k].aterrizaje)       fprintf(archivo_pagina, "&#10;Celda de aterrizaje");
    if (tabla[j][k].ataque_kamikaze)  fprintf(archivo_pagina, "&#10;Ataque kamikaze");
    if (tabla[j][k].soltar_granada1)  fprintf(archivo_pagina, "&#10;Ataque granada nº1");
    if (tabla[j][k].soltar_granada2)  fprintf(archivo_pagina, "&#10;Ataque granada nº2");
    fprintf(archivo_pagina, "&#10;Tiempo de espera: %d s\"><div
class=\"content\">%s</div></td>",
      tabla[j][k].espera,
      char_registro[j][k]
    );
  }
  fprintf(archivo_pagina, "\n\t\t\t</tr>");
}
```

Y finalmente se guarda el archivo, y el programa se devuelve al menú principal.

Uso de Inteligencia Artificial

Si bien cada día la inteligencia artificial demuestra ser una herramienta cada vez más poderosa, el equipo tomó la decisión de **no** utilizarla en lo absoluto durante el desarrollo de este proyecto.

Consideramos que en una etapa de formación y desarrollo como es un trabajo práctico para un organismo académico como es la universidad, se castiga no solo con la reducción de oportunidades para aplicar y entrenar la habilidad de resolución de problemas, habilidad fundamental en y para una ingeniería, sino también la de la incerteza e falibilidad fundamentalmente ligadas a todo modelo generativo de lenguaje (LLM) por su mismo diseño.

Al implementar código generado de manera artificial, se reduce su comprensión fundamental en el desarrollador que, en vez de redactarlo, se rebaja a reseñarlo, ya que está delegando el proceso cognitivo fundamentalmente ligado a diseñar un sistema a una máquina y solo ve el resultado ya funcional.
Esto desliga al diseñador a un mero espectador de un sistema ya funcional cuyo comportamiento comprende, más no la razón del porqué, ya que no ha reflexionado en sus implementaciones alternativas.

Parte de diseñar es tomar decisiones deliberadas, valorando múltiples opciones dados un caso y seleccionando el más adecuado según una serie de preceptos deliberados por el juicio del desarrollador.

Las inteligencias artificiales parecen ocupar cada vez más espacio en la vida cotidiana y profesional, con o sin la aprobación de a quienes se les implementa, pero este resultó no ser un momento adecuado para dicha implementación.

Debido a este y múltiples motivos personales que no tienen lugar aquí, se llegó a la decisión de **no emplearla**.

Agradecimientos

Se agradece cordialmente a Roxana Dabusti y Deix Surita por la ayuda proporcionada y el *feedback* al ofrecerse a probar el programa en busca de errores críticos al momento de su ejecución durante etapas de desarrollo.

Referencias

Manufacturer’s Guide to Developing Consumer Product Instructions - Timothy P. Smith

ANSI Escape Sequences - Christian Petersen

The C Programming Language - Brian W. Kernighan & Dennis M. Ritchie

https://cplusplus.com/, *https://stackoverflow.com/*, *https://www.geeksforgeeks.org/*, *https://en.cppreference.com/*, & *https://www.learncpp.com/*